LOGBOOK OF FMU_explore DEVELOPMENT

Here I describe the status of development the module FMU_explore. The code has been developed over long time as a gernal part of a setup script for BPL applications. Now the time has come to encapsulate this code as a central module. The simplified FMU interaction can be used for Modelica simulation models in general and not just BPL applications.

This informal text briefly describes the status and goals in near time. The idea is that this text here will complement the formal git-version handling. Now and then I plan to introduce reference to the git-version commit history.

Key architecture features are:

- Class structure of the module
- The module is then adapted to an applications through a number of variables
- The module placed in a package to simplify future use both locally and in Github

Development is mainly done in the following files:

- fmu_explore_pyfmi.py
- fmu_explore_fmpy.py
- setup TEST2_Batch.py
- setup TEST2_Batch_explore.py

I think the different examples serve as a good test bench for changes in the library. The collection of processes also is a good starting point when new systems are developed.

A general goal is:

Specific goals - now reached:

- Module v1 without simu(), show(), setLines() / Done 260819
- Module v2 complete a new concept axes as a list / Done 260819
- Module v2 now works in Windows with PyFMI / Done 260820
- Module v2 now works in Linux with PyFMI and no surprises / Done 260821
- Module v2 stored in package and used both locally and in Colab / Done 260824
- How do I make the performance function cstrProdMax() available inside simu() / Done

260825

- Make sim_res available in the Jupyter/Colab notebook when needed / Done 260829
- Make simulationTime default for simu() and also possible to set in simu() / Done 260901
- Make options in simu() possible to set / Done 260901
- Make cstrProdMax() available during plotting / Done 260901
- Make profile() available during plotting / Done 260901
- Fix simu('cont') for fmu_explore_fmpy and model_get() / Done 260907
- Make declaration of version in only one place from todays two places / Done 260909
- Made prevFinalTime local to the FMU_explore class / Done 260909
- Added to simu() the context parameter with model_get() good for eval() / Done 260909
- Added to simu() in fmu_explore_fmpy a return of sim_res / Done 260910
- Make version of scipy.optimize visible in system_info() / Done 260910
- Reorganise the list of arguments for the class (and update notebooks) / Done 260911
- Improved code: indentation, iterations in par(), init(), describe_parts() / Done 260921
- Test if sufficient to create the model just inside the module / Done 260930
- Correct check for existance of model in simu() for OMSimulato / Done 261105
-
- Handle cstrProdMax etc for fmpy
- Check describe() - commented in two applications during testing
- Take away model generation pyfmi and fmpy from the setup-file
- And generate the states in the class
- Handle default simulation time at least from OpenModelica FMUs
- Handle component_list_minimum and bring to the class

Tentative release note for version 1.3.0 say with new parameter list

Improved code quality using pylint:

- fix version() that can be retrieved in the usual way / done 1.2.1
- import of packages so that internal packages are first 1.2.2
- import only packages used, but numpy imported / done 1.2.2
- change type() to isinstance() / done 1.2.2
- change indentation to 4 spaces / done 1.2.3
- docstrings for class and functions / done 1.2.3
- modify some expression of loops over dictionaries and lists / done now 1.2.4

Day to day notes in reversed time order

Note that commit ID are now given after the date and description (i.e. opposite to Git). Far from all commits are described here though.

2026-11-05 Now I have fmu_expore_oms **version 1.2.7** working after restructuring simu()! Works for several applications. Now I can use this to make disp() work I am quite sure.

2026-11-05 Now 9 applications can run with OMSimulator with FMU_explore **version 1.2.6**. The remaining 4 applications needs more interaction and an update of the package.

2026-10-28 Now all applications are updated for both Linux and Windows using the word Appplication. Each repositoy also harmomized to a common (informal) standard.

2026-10-23 I have now updated all application notebooks for Windows using the word Application instead of Adapated for the instance of the class.

2026-10-07 I made another pre-release **version 1.2.6** and start on the next version.

2026-10-06 Now options

2026-10-05 I make a pre-release of **version 1.2.5** and start up the next version. Focus is to handle the options of simulation to facilitate the calibration application.

2026-10-05 I made the application BPL_TEST2_Batch_calibration work with oms.

2026-09-30 Now a rudimentary version of fmu_explore_oms is in place. It works for BPL_TEST2_Batch and I plan to test also on BPL_TEST2_Fedbatch soon.

2026-09-26 Started **version 1.2.5** with an early version of fmu_explore_oms.

2026-09-26 Tested in Colab BPL_TEST2_Chemostat with the older FMU done in Ubuntu 24.04 and worked fine. Then I changed the setup-file back to use the newer one made in Ubuntu 22.04. Later in the fall I think all FMUs should be recompiled.

2026-09-24 Continue to play with OMSimulator and now try to find out how to extract simulation results using Modelica variable names. Needs other packages to do. The plan forward is I think to make a rudimentary module for oms that includes par(), init(), newplot(). Then I work out simu() in the notebook and when reasonable I bring it into the module. I think simu('cont') need to wait.

2026-09-23 Started to play with OMSimulator in BPL_TEST2_Batch application. Made it work and had some interaction with Arun Palanisamy in Linköping. Made a first plot of batch growth.

2026-09-21 Now also made all docstrings and improbed code fpr iteration in par(), init(), describe_parts(). Tested in six applications. Call this **1.2.4**

2026-09-21 Now made indentation to 4 spaces instead of 3. Call this **1.2.3**. It works at least for BPL_TEST2_Batch. Also tested in Colab.

2026-09-18 Started the work yesterday with pylint to improve code quality and adopt to standards. Now call ver **1.2.2**. So far:

- import of packages not used taken away, also adjust setup-files of applications
- import order of packages, third party later
- class docstring
- change type() to isinstance() - recomended since faster

Later I will change indentation to 4 spaces instead of 3 and make that a separate comitment.

2026-09-16 I introduced ver **1.2.1** for maintenance. First thing was to bring back the posssibility to get out the version number by module._**version**_. Before that I briefly checked the need for diagrams, ax, and lines in the setup file and found that they are needed for newplot() communicating with the module functions.

2026-09-15 All applications with adjusted notebooks and run with the released version 1.2.0.

2026-09-14 Now moved defintion of statValue from setup to init-file of fmu_explore_pyfmi. Polished both modules a little bit. Now call this **ver 1.2.0**. If works in Windows I will make it my firste "release" on Github!

2026-09-11 For fmu_explore_fmpy I brought in calculation of stateValue to the module from the setup-file. Further I changed the class namde to AdaptWith and reorganized tha parameters. Call this **ver 1.1.8**

I test appications in Windows locally before I upload to Github and likely done on Monday if everthing else works fine.

2026-09-09 Now also BPL_IEC_validation works with fmu*explore*fmpy after a few fixes. But I keep the ver 1.1.6. In the notebook I have started to call the populated FMU_explore for Adpated instead of Dummy. Later I think I will change the name in the module to say AdaptWith to better bring the understanding of the workflow to the reader of the notebook.

2026-09-08 Fixed in fmu_explore_fmpy/decribe_gemeral() a model.get to model_get. Call it now **ver 1.1.6**.

2026-09-08 Now BPL_TEST2_Fedbatch and BPL_YEAST_AIR_Fedbatch_DOcontrol works with fmpy in ver 1.1.5.

2026-09-07 Now BPL_TEST2_Batch works with fmu*explore*fmpy in **ver 1.1.5**. This version of FMU_explore will be uploaded to Github.

2026-09-02 Start the work with fmu_explore_fmpy and call it ver 1.1.5

2026-09-01 Brought in the liast three applictions Chemostat, Perfusion and IEC_validation and using **FMU_explore ver 1.1.4**.

2026-09-01 Brought in BPL_TEST2_Batch_calibration.

2026-09-01 I made simu() have parameters simultionTime and options free to set but default taken from the class. Call it **FMU_explorer ver 1.1.3**.

2026-08-28 Now brought in BPL_YEAST_COB_Batch and BPL_TEST2_Batch_design_space using FMU_explore ver 1.1.2

2026-08-29 I made sim_res available outside simu() by just introducting a return statement in the function. Call this **FMU_explorer ver 1.1.2**.

2026-08-28 Now brought in BPL_IEC_Operation, BPL_STEM_AIR_Perfusion.

2026-08-27 Now brought in BPL_CHO_Perfusion*cspr*openloop.

2026-08-26 Now brought three more applications to Colab: PL_TEST2_Chemostat, BPL_YEAST_AIR_Fedbatch, BPL_CHO_Fedbatch.

2026-08-25 Almost managed to convert BPL_TEST2_Chemostat for FMU_explore 1.1.0. Remains to handle cstrProdMax() defined in user functions, though.

2026-08-24 The package updated to include import of pandas. Then run locally with BPL_TEST2_Batch, BPL_TEST2_Fedbatch and BPL_CHO_Fedbatch in Windows. The next step is to run these applications also in Colab after updating the Colab notebook. Problem today with pyfmi more general.

2026-08-21 I tested the application BPL_TEST2_Batch in Linux and worked fine. Then I made it public at Github Colab and published a note as well.

2026-08-20 I decided to let the itertool work from the module only. The function resetPen() can be used in the function newplot() in the applications script and that is the key. Also the sim_res file is for the time being just kept global from the module.

2026-08-19 Managed to get v2 working reasonbly. The key was to introduce an empty list ax, similar to diagrams, as part of the class. Then newplot() gives both of them appropirate values. Remains to fix setLines() and also simu() bring back start and final simulation time.